

Deep Learning

Final Project Report

Final Project: ASL Fingerspelling Recognition

Group: Omicron

Franco Olano Melo, 286513
Marti Ponsa Rámila, 285322

Date: 09/06/2026

1. Introduction

Sign language recognition helps break the barrier of communication between deaf and hearing people. Furthermore, ASL fingerspelling is an interesting image classification problem that enables a deep experimental comparison between different learning approaches. Our comparison objective is to address the different results that can be obtained from training a custom built Convolutional Neural Network (CNN) and well-known CNNs such as MobileNetV2 and Inceptionv3. With this experimentation, we will be able to address different models' performances, ultimately matching or improving the overall performance of known architectures.

Moreover, the project involves a second approach to the deep learning technologies by applying transfer learning on a pre-trained model to assess the introduction of new symbols. By understanding new custom fingerspelling, the experiment is able to demonstrate how anyone could apply a transfer learning mechanism to create custom handgesture commands in any system such as smart homes cameras, computer commands via camera, etc.

2. State of the Art

Image classification with deep learning has become a mature field, largely driven by convolutional neural networks (CNNs) since the introduction of AlexNet in the ImageNet Large Scale Visual Recognition Challenge 2012. Since then, architectures such as VGGNet and ResNet have shown that increasing depth, combined with techniques like batch normalization and residual connections, significantly improves performance across vision tasks.

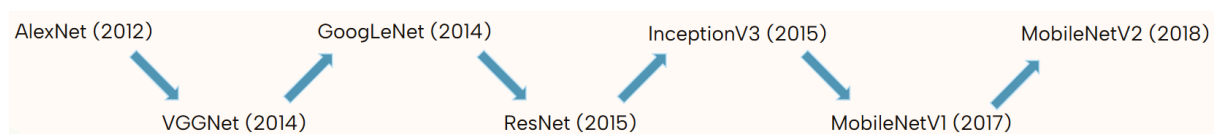
CNNs learn hierarchical feature representations, where early layers capture low-level patterns such as edges and textures, while deeper layers encode more abstract semantic information. However, training deep networks from scratch typically requires large datasets and computational resources, and models often overfit when data is limited.

To address this, transfer learning is widely used, where models pretrained on large datasets such as ImageNet are adapted to new tasks. This reduces training cost and improves generalization by leveraging previously learned representations. In this context, MobileNetV2 and InceptionV3 are two commonly used architectures. MobileNetV2 is optimized for efficiency through depthwise separable convolutions and inverted residual blocks, making it suitable for lightweight or real-time applications. In contrast, InceptionV3 uses multi-scale feature extraction through parallel convolutions, achieving higher representational power at the cost of increased complexity.

Transfer learning is typically applied either as feature extraction, where pretrained weights are frozen and only the classifier is trained, or fine-tuning, where part of the pretrained network is updated to better adapt to the target domain.

In ASL recognition, CNN-based approaches dominate modern solutions due to their ability to learn discriminative visual features directly from images. However, challenges remain due to high inter-class similarity, variations in lighting and pose, and limited subject diversity in datasets.

This project focuses on static ASL fingerspelling classification over 26 classes, later extended with three functional classes (space, delete, and nothing). The contribution lies in a systematic comparison of deep learning strategies rather than architectural novelty. We evaluate a custom CNN, MobileNetV2, and InceptionV3 under both pretrained and from-scratch settings, analyzing convergence behavior, generalization, and class-level errors, particularly between visually similar letters.



“Evolution of Convolutional Neural Network Architectures for Image Classification”

Recent advances in sign language recognition also include hybrid CNN–Transformer models, skeleton-based methods using pose estimation, and multimodal systems incorporating depth or temporal information. While these approaches improve performance in more complex settings, they fall outside the scope of this work.

3. Methodology

The dataset we use comprises static RGB images natively captured at a resolution of 200×200 pixels. The core dataset covers 26 base alphabetical classes (A–Z) representing the standard American Sign Language (ASL) fingerspelling symbols. Each class contains exactly 3,000 images, resulting in a total of 78,000 images. To enhance the real-world utility of our secondary application phase, the dataset is subsequently extended to 29 classes by integrating three functional classes: space, del (backspace), and nothing.

Unlike standard pipelines that aggressively crop or upscale visual inputs, our pipeline preserves the native dataset integrity by completely omitting spatial resizing or central cropping. The native image dimensions (200×200×3) match the initial input layers of our modified architectures exactly. Images are normalized using standard ImageNet channel statistics ($\mu=[0.485,0.456,0.406]$, $\sigma=[0.229,0.224,0.225]$) to standardize value ranges and facilitate smooth gradient updates.

Because all images from a specific subject or environment were collected sequentially in the source folder, adjacent samples have highly identical backgrounds, hand placements, and

lighting arrays. Without augmentation, a model will exploit these static environmental shortcuts instead of isolating the hand shape.

To force the networks to prioritize structural hand features, we apply a controlled set of "Safe Augmentations" to the training split only:

- Random Affine Transformations: Limited rotations ($\pm 15^\circ$) and translation shifts ($\pm 10\%$) simulate human positioning errors inside a camera frame.
- Color Jitter: Brightness and contrast shifts (± 0.2) decouple the hand profile from fixed illumination setups.

Horizontal flipping and large rotations were excluded from the augmentation strategy because they may distort the semantic meaning of ASL gestures. Such transformations can produce unrealistic hand configurations that are not representative of valid signs, leading to incorrect training samples. As future work, a dedicated dataset containing both original and horizontally mirrored samples could be created to enable reliable recognition of signs performed with either the left or right hand.

The data is partitioned using a randomized strategy with a fixed seed (42) into a 70% Training, 15% Validation, and 15% Testing structure.

Since the dataset does not provide subject identifiers, a true subject-independent train-validation split could not be performed. Consequently, samples from the same individual may appear in both sets, potentially leading to slightly optimistic validation results. To reduce this effect, all class directories were randomly shuffled before partitioning.

The experiments were conducted on the Kaggle platform, which was selected for its larger storage capacity and longer execution times, enabling efficient training and logging of the models.

To answer the core question of this project: which models work best for ASL fingerspelling recognition. We compare three types of architectures.

3.1. Architecture A: Custom CNN (ASLCustomCNN)

Our baseline model is a custom convolutional neural network composed of four convolutional blocks followed by a classification head.

The architecture progressively increases the number of channels ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), allowing the network to learn hierarchical features, from simple edges to more complex hand structures.

Each block consists of:

- Convolution (3×3 kernel, padding=1)

- Batch normalization
- ReLU activation
- Max pooling (2×2)

After feature extraction, an adaptive average pooling layer reduces the spatial dimensions to a fixed-size representation. A dropout layer ($p=0.5$) is applied before the final linear classifier to reduce overfitting.

This architecture is intentionally simple and lightweight, making it suitable as a controlled baseline for comparison.

3.2. Architecture B: MobileNetV2 (ASLMobileNetV2)

MobileNetV2 is a lightweight architecture designed for efficiency, particularly in mobile and real-time applications.

It is based on depthwise separable convolutions and inverted residual blocks, which significantly reduce the number of parameters while maintaining strong performance.

We evaluate MobileNetV2 in two configurations:

- From scratch, to assess its ability to learn task-specific features
- Feature extraction, where pretrained ImageNet weights are used and only the final classifier is trained

3.3. Architecture C: InceptionV3 (ASLInceptionV3)

InceptionV3 is a deeper and more complex architecture that processes information at multiple spatial scales using parallel convolutional filters.

This design is particularly suitable for tasks where small spatial differences are important, such as distinguishing similar hand gestures.

As with MobileNetV2, we evaluate:

- Training from scratch
- Feature extraction using pretrained weights

To ensure a fair comparison, all models were trained under the same conditions using CrossEntropyLoss, a batch size of 10, five training epochs, and identical data splits and augmentation settings. Two optimization strategies were evaluated: SGD with momentum and Adam, allowing us to analyze both architectural and optimization-related differences.

Model performance was evaluated using accuracy, macro F1-score, per-class F1-scores, and confusion matrices. While accuracy provides an overall measure of performance, the

additional metrics help identify weaknesses in specific classes and highlight common misclassifications between visually similar ASL gestures.

4. Experiments

The experimental design focuses on a controlled comparison between different deep learning strategies for ASL fingerspelling recognition, with emphasis on architectural differences, convergence behavior, and generalization performance rather than final accuracy alone.

To ensure fairness, all models are trained under identical experimental conditions, including the same preprocessing pipeline, data partition, optimizer choice, batch size, and training schedule. The evaluation is performed using multiple complementary metrics, including accuracy, macro-averaged F1-score, per-class precision and recall, and confusion matrices. Macro F1 is particularly important in this context, as it provides a balanced measure of performance across all 26 classes and highlights weaknesses in visually similar gestures.

The experiments are divided into two phases. The first phase performs a strictly controlled comparison between architectures, evaluating a custom CNN, MobileNetV2, and InceptionV3 under the same training setup using pretrained weights in feature extraction mode. This phase isolates architectural impact under constrained adaptation and allows analysis of convergence speed, stability, and the effectiveness of pretrained representations for fine-grained hand gesture recognition.

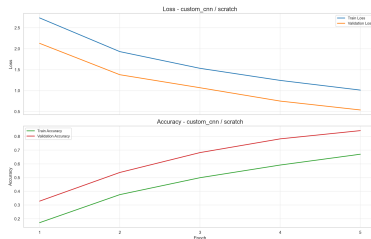
The second phase builds on the results of the first phase by selecting the best-performing model and training configuration, which corresponds to MobileNetV2 trained from scratch. This model is then further trained and extended to incorporate the additional three functional classes (space, delete, and nothing), expanding the task from 26 to 29 classes. This phase evaluates the model's ability to adapt to new categories while preserving previously learned representations, effectively simulating a realistic incremental deployment scenario.

Overall, this experimental setup separates architectural effects from task expansion effects, enabling a clearer interpretation of performance gains and model adaptability.

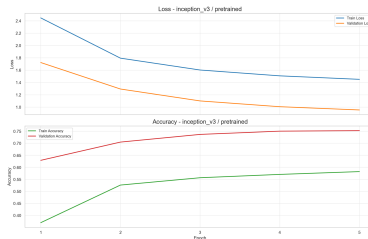
5. Results

This section presents the results of all experiments, including performance metrics, confusion matrices, and per-class F1 scores.

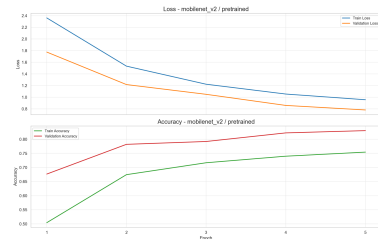
Training loss, validation loss, training accuracy and validation accuracy for all experiments:



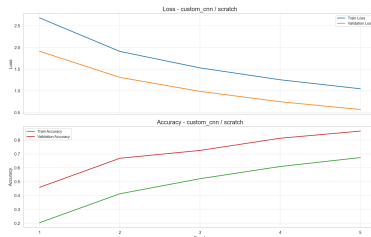
Custom - SDG
Loss: 0.5308
Accuracy: 0.8459



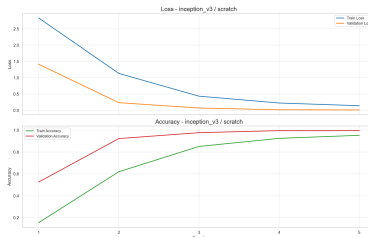
InceptionV3 - FE
Loss: 0.9609
Accuracy: 0.7511



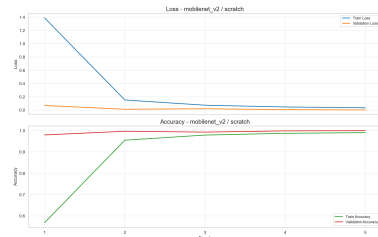
MobilenetV2 - FE
Loss: 0.7909
Accuracy: 0.8219



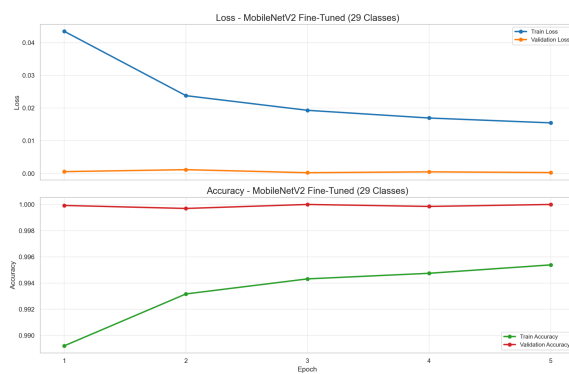
Custom - Adam
Loss: 0.5607
Accuracy: 0.8716



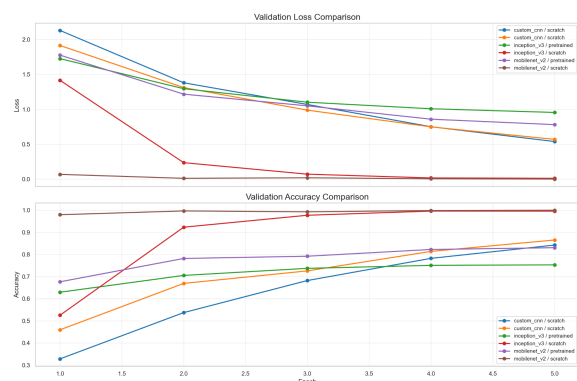
InceptionV3 - Full
Loss: 0.0178
Accuracy: 0.9958



MobilenetV2 - Full
Loss: 0.0013
Accuracy: 0.9997

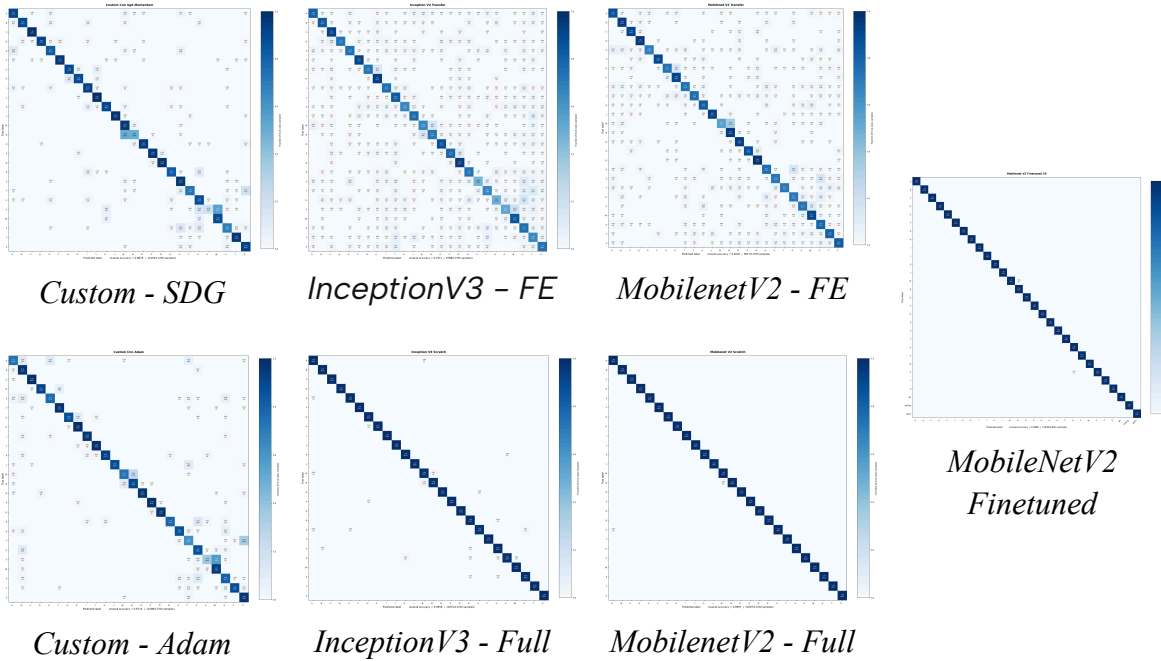


MobileNetV2 Finetuned
Loss: 0.0007
Accuracy: 0.9998

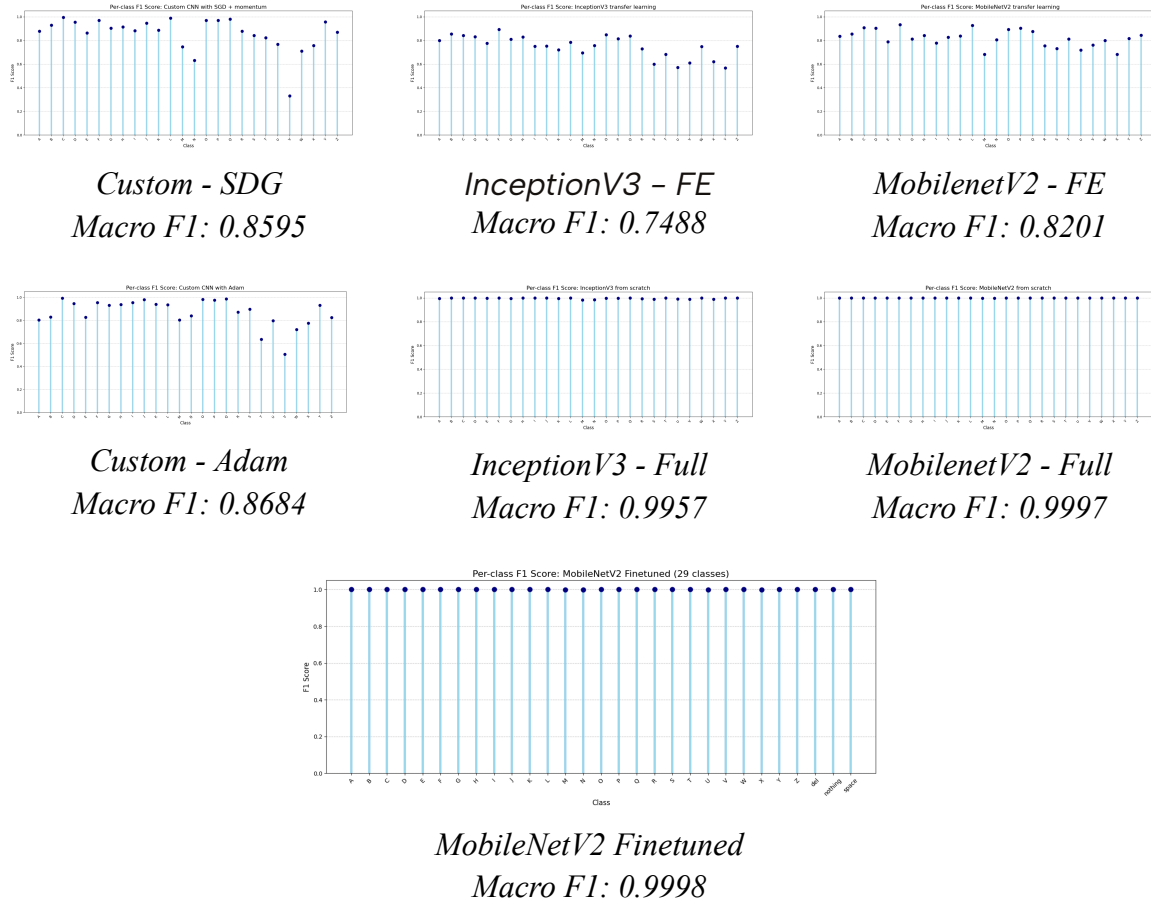


*Comparison of Validation Accuracy and
Validation Loss*

Confusion Matrices for all experiments:



Per-class F1 score for all experiments:



6. Discussion

The experimental results reveal clear differences between architectures, training strategies, and optimization methods, particularly under the constrained training regime of five epochs.

A first important observation is the poor performance of feature extraction approaches, particularly for InceptionV3. Since only a small fraction of the network is trained, the models are unable to adapt pretrained ImageNet features to the specific characteristics of ASL gestures, which depend on fine-grained hand geometry. In contrast, when these architectures are trained from scratch, they achieve near-perfect performance, indicating that the dataset is sufficiently large for deep models to learn highly discriminative features.

The custom CNN, despite having significantly fewer parameters, achieves competitive results and outperforms both feature extraction models. This suggests that a simpler architecture trained directly on the target data can be more effective than relying on pretrained representations with limited adaptation.

Regarding optimization, Adam consistently outperforms SGD with momentum. This is mainly due to its faster convergence, which becomes particularly relevant given the small number of training epochs.

From the convergence curves, it can be observed that larger architectures, especially MobileNetV2, reach very low loss values within the first epoch. While this indicates that the task is relatively well-structured, it should be interpreted carefully, as the absence of subject-independent splits may lead to optimistic performance estimates.

Error analysis through confusion matrices and per-class F1 scores shows that most misclassifications occur between visually similar letters, particularly M-N and U-X. These errors highlight the intrinsic difficulty of distinguishing subtle differences in hand pose, even for highly accurate models.

Finally, the extension to 29 classes demonstrates that the model is able to incorporate new categories without degrading performance. By reusing previously learned weights, the model avoids catastrophic forgetting and quickly adapts to the new classes, especially since they are visually distinct.

Overall, the results indicate that training deep architectures from scratch is highly effective for this task, while feature extraction is insufficient due to domain mismatch. However, the lack of subject-independent evaluation remains a limitation that should be considered when interpreting the results.

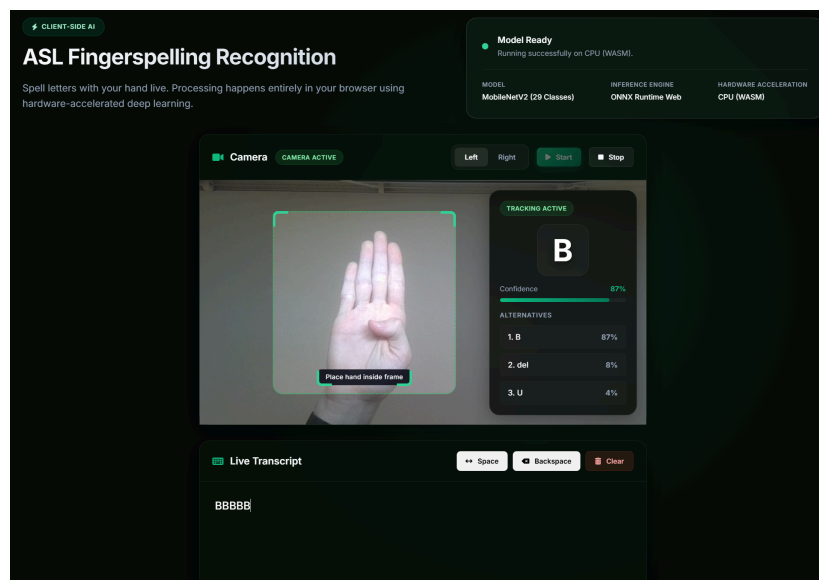
7. Demo Application

As a complementary component to the experimental evaluation, a simple real-time application was developed to showcase the model's predictions.

The system uses a webcam input to capture hand gestures and performs inference using a selected trained model. The predicted classes are displayed in real time, allowing the user to construct words through ASL fingerspelling. Additional classes such as space, delete, and nothing were introduced to improve interaction and enable basic text editing functionality.

The application demonstrates that the trained models, particularly MobileNetV2, are capable of operating in a real-time setting with high accuracy. However, this component is intended only as a proof of concept, and does not replace the quantitative evaluation presented in the previous sections.

The following image shows an example of the interface during execution.



Real-time ASL recognition demo interface.

8. Conclusions

The experiments show that feature extraction with frozen pretrained models is not well suited for ASL fingerspelling, likely due to a domain gap between ImageNet features and fine-grained hand gestures. In contrast, models trained from scratch achieved stronger performance across configurations.

The custom CNN, despite having significantly fewer parameters than MobileNetV2, delivered competitive results, highlighting that task-specific architectures can be highly effective for this problem.

Adam consistently outperformed SGD with momentum, mainly due to faster convergence within the limited five-epoch training setup. Small differences between MobileNetV2 and InceptionV3 can be linked to architectural factors such as receptive field design and its effect on capturing fine spatial details.

Finally, extending MobileNetV2 to 29 classes through fine-tuning preserved performance on the original classes while successfully integrating new ones, showing no signs of catastrophic forgetting and demonstrating good transferability of learned features.

Overall, the results indicate that architectural design and optimization strategy have a greater impact than model size alone in ASL classification tasks.

9. References

1. Mohammed-Shoaib. (s. f.). GitHub - Mohammed-Shoaib/ASL-Fingerspelling: Detection of the alphabets in American Sign Language (ASL). GitHub. <https://github.com/Mohammed-Shoaib/ASL-Fingerspelling>
2. Guillaume phd. (s. f.). GitHub - guilleumephd/deep_learning_hand_gesture_recognition: A deep learning model for hand gesture recognition. GitHub. https://github.com/guilleumephd/deep_learning_hand_gesture_recognition
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, & Liang-Chieh Chen. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. arXiv. <https://arxiv.org/abs/1801.04381>
4. Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, & Zbigniew Wojna. (2015). Rethinking the inception architecture for computer vision. arXiv. <https://arxiv.org/abs/1512.00567>
5. Akash Nagaraj. (2018). ASL Alphabet [Dataset]. Kaggle. <https://doi.org/10.34740/KAGGLE/DSV/29550>
6. Pannattee, P., Kumwilaisak, W., Hansakunbuntheung, C., Thatphithakkul, N., & Kuo, C. J. (2023). American Sign language fingerspelling recognition in the wild with spatio temporal feature extraction and multi-task learning. *Expert Systems With Applications*, 243, 122901. <https://doi.org/10.1016/j.eswa.2023.122901>